Week 3 Deliverable · AI-Assisted React Application Build

Application: MindGuard AI · Front-End AI Companion & Mental Health Triage Interface

FlyRank AI Internship

Track: Front-End AI Engineering

1. OVERVIEW OF THE COMPLETED APPLICATION

MindGuard AI Interface Architecture

A production-ready React 18 application built with Vite and Lucide Icons demonstrating four core front-end AI engineering features:

- **Token Streaming & Latency Simulation:** Real-time Server-Sent Events (SSE) token generation pipeline with sticky auto-scroll and Time to First Token (TTFT) counter (120ms streaming vs. 1,800ms legacy blocking benchmark).
- **3-Action Acute Crisis Triage Modal:** Consolidates emergency options down to 3 acute triage choices (Call 988, Text 741741, Reach Contact) based on Hick's Law, with a 1-click clipboard desktop fallback.
- **WCAG 2.1 AA Model Settings Drawer:** Accessible configuration drawer for Anthropic API keys with RFC email validation, password masking toggle, and submit error focus management.
- **Telemetry & Audit Inspector:** Live telemetry panel showing TTFT, token speed (t/s), and real-time NLP classification states.

2. PROMPTS USED DURING DEVELOPMENT

Prompt 1: Streaming Chat Architecture & State Pipeline

```
Act as a Senior React Engineer. Scaffold a streaming AI chat interface in React 18 with message state
(role, timestamp, intent badge), simulated token streaming with configurable TTFT latency (120ms stream vs
1,800ms blocking), and sticky auto-scroll locking to bottom during generation without snap-backs.
```

Prompt 2: Crisis Triage Modal (Hick's Law & Desktop Fallback)

```
Design an accessible Emergency Crisis Modal in React applying Hick's Law: 3 primary actions (Call 988,
Text 741741, Reach Contact), 1-click clipboard copy fallback for desktop tel: links, progressive
disclosure for secondary hotlines, and role="dialog" accessibility.
```

Prompt 3: Production Settings Form & Validation Gate

```
Build an accessible Settings Form in React with real-time onBlur/onSubmit validation for RFC email regex
and Anthropic API key format, string .trim() sanitization, password type="password" with type="button"
show/hide toggle, and WCAG 2.1 AA focus shifting on failed submit.
```

3. HOW AI ASSISTED THROUGHOUT THE IMPLEMENTATION

AI accelerated the development lifecycle by generating component boilerplates for complex multi-tier state orchestration (streaming token intervals, latency timers, and intent badges) in minutes. It also formulated accurate regular expressions for RFC email schemas and Anthropic key prefix patterns (`^sk-ant-api[0-9a-zA-Z_-]{20,}$`), and systematically mapped WCAG 2.1 AA ARIA tags across all form controls.

4. MANUAL IMPROVEMENTS, CORRECTIONS & REFACTORING

Fix 1: Accidental Form Submit Bug (Button Type Omission)

AI Flaw: Omitted `type="button"` on the password visibility toggle. Because HTML buttons default to `type="submit"`, clicking "Show" accidentally submitted the form.

```
- <button onClick={() => setShowKey(!showKey)}>
+ <button type="button" onClick={() => setShowKey(!showKey)}>
```

Fix 2: Whitespace Validation Bypass (Sanitization)

AI Flaw: Used naive length check `value.length >= 2`, allowing whitespace strings (" ") to bypass validation.

```
- if (!value || value.length < 2)
+ const trimmed = typeof value === 'string' ? value.trim() : value;
+ if (!trimmed || trimmed.length < 2)
```

Fix 3: Desktop `tel:` Broken Link Fallback

AI Flaw: Bare `<a href="tel:988">` links failed completely on desktop browsers without soft fallbacks.

```
+ Implemented secondary desktop "Copy" button triggering navigator.clipboard.writeText('988') + toast.
```

5. RUBRIC VERIFICATION CHECKLIST

- ✓ Completed React application in Week3/
- ✓ Prompts recorded for all key components
- ✓ AI assistance explained clearly
- ✓ 3 manual refactoring diffs documented